1. Klassdesign

InMemoryDeviceRepository lagrar och skyddar en samling av enheter i en arraylist. Listan skyddas genom att alltid skicka ut en oföränderlig kopia av listan, vilket gör att listans struktur skyddas och ingen utifrån kan använda add/remove/clear på den faktiska listan. Enheterna skyddar sig själva med privata fält och egna regler. Alla enheter i lagringen måste ha unika id för identifiering, annars kastas ett exception, men de kan ha samma namn då det bara är en “etikett”.

SmartDeviceService tar hand om affärsregler och validerar indatan innan repositoryt kan anropas. T.ex. om man vill lägga till en enhet så kastas ett exception om indatan är null. Klassen tar emot interfacet DeviceRepository istället för klassen InMemoryDeviceRepository för att vara mer flexibel för framtida implementationer och för att kunna mocka tester exempelvis med Mockitos verify kan jag kontrollera att repositoryt inte anropas när indatan är ogiltig.

Jag valde att dela upp det enligt SRP (Single Responsibility Principle) så att varje klass skulle ha sitt eget ansvar. Det gör att koden blir enklare att hantera i framtiden om den skulle behöva byggas ut,Om repositoryt behöver ändras, ändrar jag bara i InMemoryDeviceRepository och behöver jag ändra validering så ändrar jag i SmartDeviceService. Om allt hade varit i en stor klass hade jag inte kunnat testa valideringen isolerat med en mock då lagringen är inbyggd i klassen. Ett exempel på SRP är metoden “getDeviceList()” som låg i SmartDeviceService tills jag implementerade ReportService som har ansvar för rapportering, då flyttade jag metoden dit istället (den fick också ett nytt namn listAllDevices()).

2. Arv eller komposition

Arv finns i form av att Lamp, Camera och Radiator alla ärver från SmartDevice. SmartDevice är en abstrakt klass som bara fungerar som en föräldraklass. Lamp är en SmartDevice, klassen är abstrakt eftersom det inte finns någon generell enhet, bara konkreta typer. Jag valde att göra så då alla tre subklasserna är en typ av SmartDevice som har många gemensamma metoder, tillstånd och beteenden, t.ex. getId(), isActive(), id, namn, aktiv-status och tidsstämplar. Då subklasserna ärver metoder från föräldern behöver jag bara skriva metoder som är specifika för subklasserna i varje subklass t.ex. toggleRecording(). Att alla subklasser räknas som SmartDevices (polymorfism) är också bra då jag enkelt kan lagra alla i samma repository. Exempel på komposition i koden är att SmartDevice har en klocka, den skickas in i konstruktorn och används för att göra tester förutsägbara. Clock.fixed() gör att jag kan manipulera tiden för att se hur koden reagerar vid olika tillfällen t.ex. om jag vill se att getLastActivation() eller getLastDeactivation() fungerar.

Jag hade kunnat skippa arv och använt en Device-klass och använt enum för DeviceType. Det hade brutit mot SRP då klassen har ansvar för olika typer av enheter. En lampa behöver inte ha isRecording() eller targetTemperature() och ett element behöver inte brightness. Det hade också brutit mot OCP då det inte går att lägga till ny funktionalitet utan att ändra befintlig kod. Med arv kan jag bara lägga till en ny klass som ärver från SmartDevice utan att röra någon annan kod. Jag hade också kunnat skippa klock-kompositionen och använt mig av LocalDateTime.now() men då hade jag inte kunnat förutsäga tidsstämplarna i testerna.

3. Interface

DeviceRepository är ett interface som InMemoryDeviceRepository implementerar och serviceklasserna har ett kontrakt med. Metoderna beskriver vad som kan göras med lagringen, t.ex. addDevice(), removeDevice() och findDeviceById(). Om jag behöver lägga till en ny form av lagring kan jag enkelt implementera DeviceRepository vilket är bra för OCP. Jag behöver inte ändra kod, bara lägga till. Kontraktet mellan DeviceRepository och serviceklasserna gör att servicerna bara känner kontraktet och inte hur enheterna lagras. Om jag bara hade haft InMemoryDeviceRepository utan interface så hade servicerna tagit InMemoryDeviceRepository i konstruktorn och byte av lagring hade krävt ändring i båda servicerna. Testerna hade också körts mot en riktig lista istället för en isolerad mock. Med en mock kan jag kontrollera att repositoryt inte anropas vid ogiltig indata vilket inte går med en riktig lista. Interfacet Activatable används inte polymorft idag men skulle kunna komma till användning i framtiden.

4. SOLID

Open/Closed Principle (OCP). Principen säger att koden ska vara öppen för utökning men stängd för ändring. Jag har använt mig av arv och polymorfism för att göra det enkelt att bygga ut koden utan att behöva ändra befintlig kod. Att ändra befintlig kod riskerar att förstöra fungerande och redan testad kod. OCP undviker det. Servicerna och repositoryt arbetar mot den abstrakta klassen SmartDevice i stället för subklasserna (polymorfism). Därför kan jag lägga till en ny enhetstyp genom att ärva från SmartDevice, utan att ändra någon befintlig klass.

5. Vidareutveckling (tio nya enhetstyper)

Det är enkelt att lägga till tio nya enhetstyper eftersom jag bara behöver skapa en ny klass som ärver från SmartDevice. Det som sedan hade behövts göras är att lägga till klassens specifika metoder, toString() och tester.

Något som har förbättringspotential är List och findDeviceByName. List skulle kunna vara en Map istället. Det hade gjort sökningen via id och att skapa nya enheter snabbare då hela listan inte behöver loopas igenom för att hitta rätt enhet eller kontrollera om id redan finns, en Map hade direkt hittat matchande id via nyckeln. List räcker just nu då programmet inte är så stort. Eftersom namn inte är unika returnerar findDeviceByName bara den första träffen, för att få alla träffar kan metoden returnera en List<SmartDevice> med alla träffar.

Viktor Parment